

AgriGuard AI

Smart Crop Disease and Risk Prediction System

Week 4 – Implementation Strategy and Code Architecture Outline

Python + Machine Learning | Implementation Planning Document

1. Objective

Week 4 converts the AgriGuard AI design developed in the earlier project stages into an implementation-ready Python architecture. The proposed system will ingest labelled crop-leaf images, validate the data, apply a consistent preprocessing pipeline, train a disease-classification model, evaluate it using multiple metrics, analyse errors, and then use the saved model for disease prediction and a rule-based risk level. The design separates these responsibilities into modules so that data processing, model training and prediction can be tested or modified independently. This is important because a machine-learning project should not depend on one large script. Configuration, preprocessing, model logic, evaluation, logging and inference should have clear interfaces. The implementation also records the dataset version, random seed, hyperparameters, model artifact and evaluation results so that experiments can be reproduced.

2. Concrete Dataset Evidence and Implementation Assumptions

PlantVillage is used as the reference dataset for the implementation plan. TensorFlow Datasets documents PlantVillage as 54,303 healthy and unhealthy leaf images divided into 38 categories by species and disease. The documented feature structure contains an image and an integer class label with 38 classes. This makes it suitable for designing the AgriGuard AI disease-classification pipeline. The dataset should be treated as a benchmark/reference source rather than proof of real-world field performance because published work notes that PlantVillage images were collected under controlled conditions.

Reference evidence: TensorFlow Datasets reports 54,303 images and 38 categories. A plant-disease study also describes the dataset as 54,305 images across 38 crop-disease classes and notes its controlled image conditions. The small count discrepancy is version/reporting related, so this report uses 54,303 as the implementation reference documented by TFDS.

Proposed implementation split (engineering plan, not an observed result):

- 70% training: approximately 38,012 images.
- 15% validation: approximately 8,145 images.
- 15% test: approximately 8,146 images.
- Use stratification where appropriate so class proportions are preserved.
- Do not apply augmentation to validation/test samples; augmentation is training-only.
- Maintain the original test set untouched until final evaluation.

3. Hypothesis Formulation

Primary hypothesis (H1): A consistent preprocessing pipeline combined with realistic training-data augmentation will improve generalization of the AgriGuard AI disease classifier compared with a baseline pipeline without augmentation.

Null hypothesis (H0): Adding preprocessing/augmentation does not produce a meaningful improvement in validation/test performance.

Engineering success target: achieve a macro F1-score of at least 0.90 on the held-out test set. This is a target for experimentation, not a claimed achieved result. The final report should replace this target with the measured value after implementation.

Secondary hypothesis: monitoring class-wise recall and F1 will reveal weak disease classes that overall accuracy can hide. Therefore model selection will consider macro/class-wise performance rather than accuracy alone.

4. High-Level Architecture

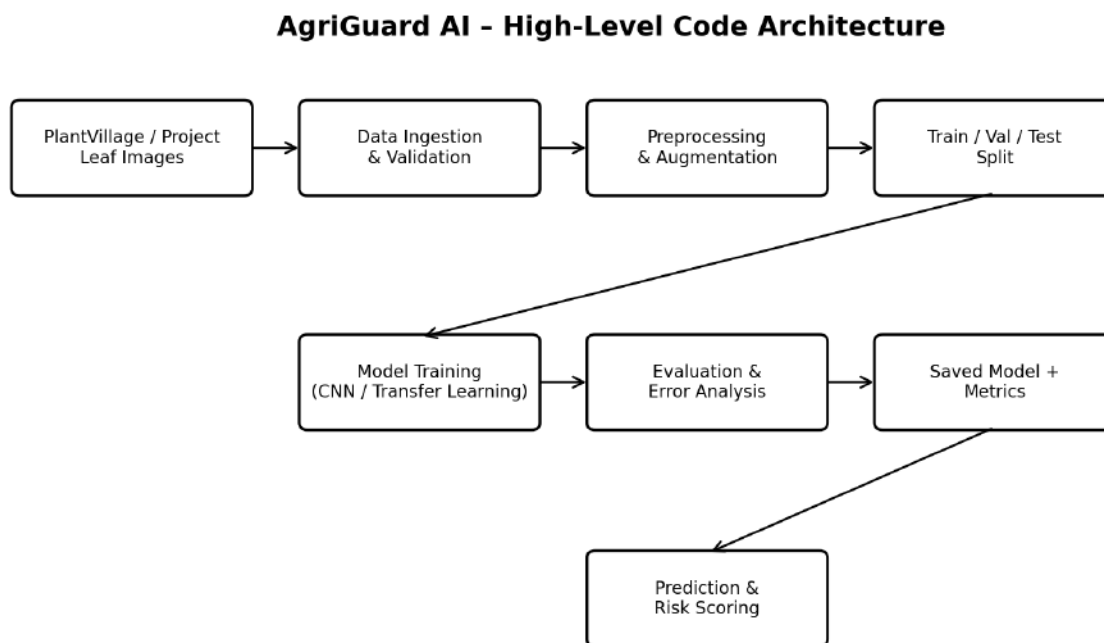


Figure 1. Proposed AgriGuard AI implementation architecture.

5. End-to-End Workflow

AgriGuard AI - End-to-End Implementation Workflow

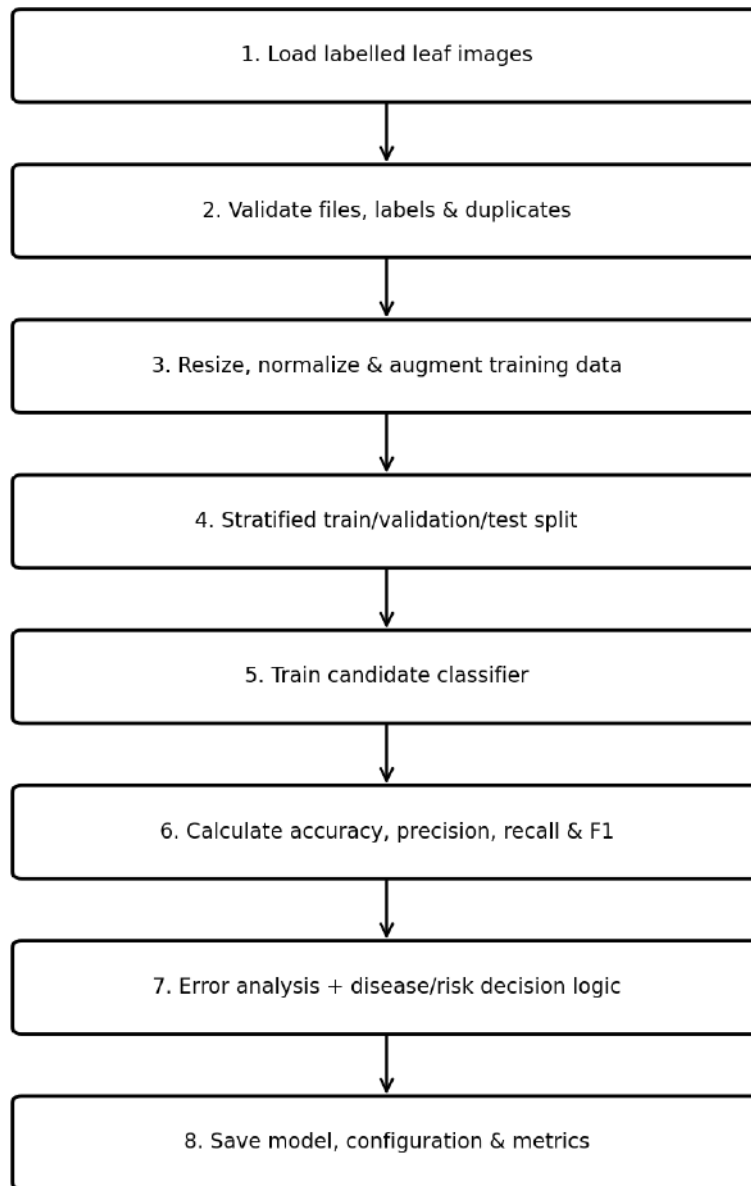


Figure 2. Proposed implementation workflow from data ingestion to saved model.

6. Python Project Structure

```
agriguard-ai/  
├── data/  
│   ├── raw/  
│   └── processed/  
├── src/  
│   ├── config.py  
│   ├── data_loader.py  
│   ├── validation.py  
│   ├── preprocessing.py  
│   ├── model.py  
│   └── train.py
```

— evaluate.py — risk_engine.py — predict.py — utils.py — tests/ — models/ — reports/ — requirements.txt — README.md

7. Module and Function Breakdown

Module / Function	Responsibility	Input	Output
data_loader.py / load_dataset()	Discover images and labels.	Dataset path	Records
validation.py / validate_dataset()	Check missing/corrupt files, labels and duplicates.	Records	Validation report
preprocessing.py / preprocess()	Resize, normalize and prepare model input.	Image	Model-ready tensor
preprocessing.py / augment()	Apply training-only transformations.	Training image	Augmented image
model.py / build_model()	Create selected CNN/transfer-learning classifier.	Config	Model
train.py / train_model()	Train, validate, checkpoint and log.	Train/validation data	Best model + history
evaluate.py / evaluate_model()	Compute metrics and confusion matrix.	Model + test data	Metrics/report
risk_engine.py / calculate_risk()	Convert prediction/confidence and disease class into project risk level.	Prediction + confidence	Low/Medium/High risk
predict.py / predict()	Load artifact and run the same preprocessing for inference.	New image	Class + confidence + risk
utils.py	Logging, seed setup and reusable helpers.	Runtime info	Utilities

8. Core Pseudocode

8.1 Data ingestion and validation

<pre> FUNCTION load_dataset(path): records = [] FOR each file: IF supported image: read class label append (file_path, label) RETURN records FUNCTION validate_dataset(records): check file exists and can be decoded check label belongs to approved class list detect duplicates / suspicious records calculate class counts generate validation report IF critical errors: </pre>

```
STOP pipeline
RETURN valid_records
```

8.2 Preprocessing and training

```
FUNCTION preprocess(image, training):
    resize image to target size
    convert to required color format
    normalize pixel values
    IF training:
        apply realistic augmentation
    RETURN image

FUNCTION train_model(train, validation, config):
    set random seeds
    model = build_model(config)
    FOR epoch in range(config.epochs):
        train one epoch
        calculate training metrics
        evaluate validation data
        save best checkpoint
    RETURN best_model, history
```

8.3 Evaluation

```
FUNCTION evaluate_model(model, test_data):
    predictions = model.predict(test_data)
    predicted_labels = argmax(predictions)

    accuracy = accuracy_score(true, predicted_labels)
    precision = precision_score(true, predicted_labels, average='macro')
    recall = recall_score(true, predicted_labels, average='macro')
    f1 = f1_score(true, predicted_labels, average='macro')
    matrix = confusion_matrix(true, predicted_labels)

    save all metrics and matrix
    inspect high-frequency false positives/false negatives
    RETURN metrics, error_report
```

8.4 Disease and risk prediction

```
FUNCTION calculate_risk(predicted_class, confidence):
    IF confidence >= 0.90:
        confidence_level = "high"
    ELSE IF confidence >= 0.70:
        confidence_level = "moderate"
    ELSE:
        confidence_level = "low"

    combine confidence_level with disease severity mapping
    RETURN risk_level

FUNCTION predict(image):
    load saved model and preprocessing configuration
    processed = preprocess(image, training=False)
    class_probabilities = model.predict(processed)
    class_id = argmax(class_probabilities)
    confidence = max(class_probabilities)
    risk = calculate_risk(class_id, confidence)
    RETURN class_id, confidence, risk
```

Risk scoring is an application-layer decision rule in this Week 4 design. It should not be presented as a medically/agronomically validated disease-severity measurement until it has been tested against expert-labelled field outcomes.

9. Evaluation Plan and Concrete Metrics

The evaluation will report overall and class-wise results. Scikit-learn's classification metrics include accuracy, precision, recall, F1-score and confusion matrices. F1 is the harmonic balance of precision and recall, so it is useful when one class should not dominate the assessment.

- Accuracy = correctly classified samples / total samples.
- Macro precision = average precision across all disease/healthy classes.
- Macro recall = average recall across all classes.
- Macro F1 = average F1 across classes; proposed primary model-selection metric.
- Confusion matrix = identifies which disease classes are being confused.
- Error analysis = inspect representative false positives and false negatives from weak classes.
- Compare baseline versus augmented model using the same held-out test set.

The implementation will use the scikit-learn metrics API for standard classification reporting. Source: scikit-learn documentation.

10. Maintainability, Version Control and Testing

- Follow a single-responsibility design: each module should have one major responsibility.
- Use descriptive names, docstrings and type hints for important functions.
- Keep hyperparameters and file paths in configuration rather than hard-coding them.
- Use logging to record dataset version, seed, configuration, training progress and metrics.
- Keep raw data separate from processed data and never overwrite the original dataset.
- Use Git/GitHub with focused commits and a README describing how to reproduce the experiment.
- Pin important dependencies in requirements.txt and record the Python environment.
- Unit-test validation and preprocessing; integration-test the complete prediction path.
- Save model weights, class mapping, preprocessing configuration and metrics together.

11. Generic Week 4 Timeline

Day	Implementation Activity	Deliverable / Evidence
Day 1	Create folders, config and environment.	Project structure + requirements.txt
Day 2	Implement data ingestion and validation.	Validation report / sample checks
Day 3	Implement preprocessing and augmentation.	Preprocessing module + tests
Day 4	Implement model training and checkpoints.	Training script + saved checkpoint
Day 5	Implement metrics, confusion matrix and error analysis.	Evaluation report
Day 6	Implement prediction + risk engine and integration tests.	Inference test result
Day 7	Documentation, cleanup, GitHub	README + Week 4 documentation

12. Risk Assessment and Mitigation Strategy

Risk	Potential Impact	Priority	Mitigation
Corrupt/mislabeled images	Incorrect learning and unreliable metrics.	High	Validate files/labels before training; log rejected samples.
Class imbalance	Poor recall for less represented diseases.	High	Use stratified splits and macro/class-wise metrics; consider class weighting.
Data leakage	Artificially high test performance.	High	Keep test set isolated; apply augmentation only to training data.
Overfitting to controlled backgrounds	Poor field performance despite strong benchmark score.	High	Use augmentation and, where possible, validate on diverse field images; report dataset limitations.
Dependency conflicts	Training/inference failures.	Medium	Use requirements.txt, version recording and clean-environment testing.
Limited compute	Long training or memory errors.	Medium	Use smaller batches, efficient transfer learning and staged experiments.
Training/inference mismatch	Incorrect predictions on application inputs.	High	Reuse identical preprocessing code and store its configuration with the model.
Risk-score overinterpretation	Users may treat confidence as guaranteed disease severity.	High	Label risk as model/application output; validate against expert data before deployment.
Integration/API errors	Wrong image shape or class mapping.	Medium	Add schema checks and end-to-end prediction tests.
Poor reproducibility	Experiments cannot be compared reliably.	Medium	Fix seed, record configuration, dataset version, metrics and artifact.

13. Expected Outcome

By completing this implementation strategy, AgriGuard AI will have a clear path from labelled crop-leaf data to disease classification and risk-level output. The design provides concrete dataset assumptions, a testable hypothesis, measurable evaluation criteria, modular Python responsibilities, pseudocode, a generic implementation timeline and risk controls. Importantly, the proposed 0.90 macro-F1 threshold is an engineering target rather than a fabricated result. Actual performance must be inserted only after the model is trained and evaluated on the held-out test data. The architecture also separates benchmark disease recognition from the later risk-scoring layer, making it possible to improve either component without rewriting the complete application.

14. References

1. TensorFlow Datasets. PlantVillage dataset documentation: 54,303 healthy/unhealthy leaf images, 38 categories, image + class-label structure. https://www.tensorflow.org/datasets/catalog/plant_village
2. Scikit-learn. Metrics and scoring documentation: classification metrics including accuracy, precision, recall, F1 and confusion matrix. https://scikit-learn.org/stable/modules/model_evaluation.html
3. Ferentinos, K. P. (2018). Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture*, 145, 311–318. <https://doi.org/10.1016/j.compag.2018.01.009>
4. Sladojevic, S. et al. (2016). Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification. *Computational Intelligence and Neuroscience*. <https://doi.org/10.1155/2016/3289801>